ECE 3218

Lab #1 Report

Daniel Xu

September 10, 2026

Overview

This lab aims to teach about the usage of Microchip Studio by writing assembly programs and using the built-in simulator and debugger. The pre-lab was completed with flow charts in my notebook and programs were written before coming to lab.

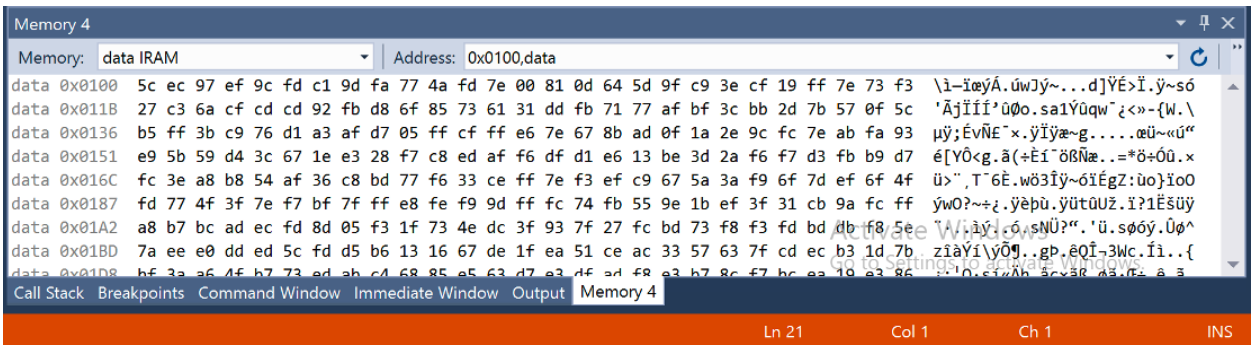
Procedure

1. The Arduino board was connected to the Atmel-ICE using the 6-pin ribbon cable and connected to power and our computer using the provided USB hub.
2. Microchip Studio was set up to program the board through the Device Programming window
3. The assembly code from the pre-lab was compiled and uploaded to the board.
4. The debugger was setup by turning on debug WIRE mode. This required toggling the power of the board after turning debugWIRE on.
5. Each of the three pre-lab programs were run using two different data sets. The results were recorded and compared to the expected results.

Results

Images show memory and registers at the end of the program. Output data is located at 0x110 unless otherwise specified.

Program 1 (counting number of items in a list)



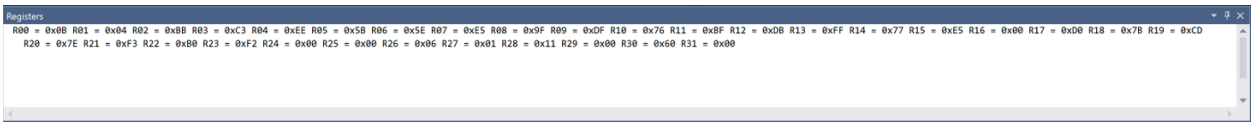


Figure 1. Output data of program 1, test 1. 13 entries, result is 0x0d = 13.

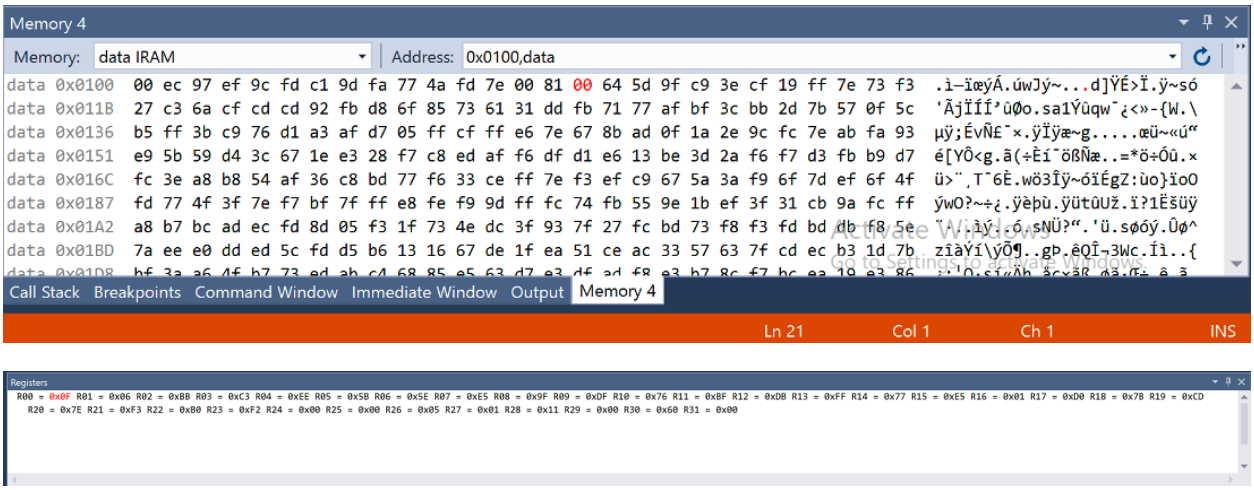


Figure 2. Output data of program 1, test 2. 0 entries, result is 0x00 = 0.

Program 2 (max value in a list)

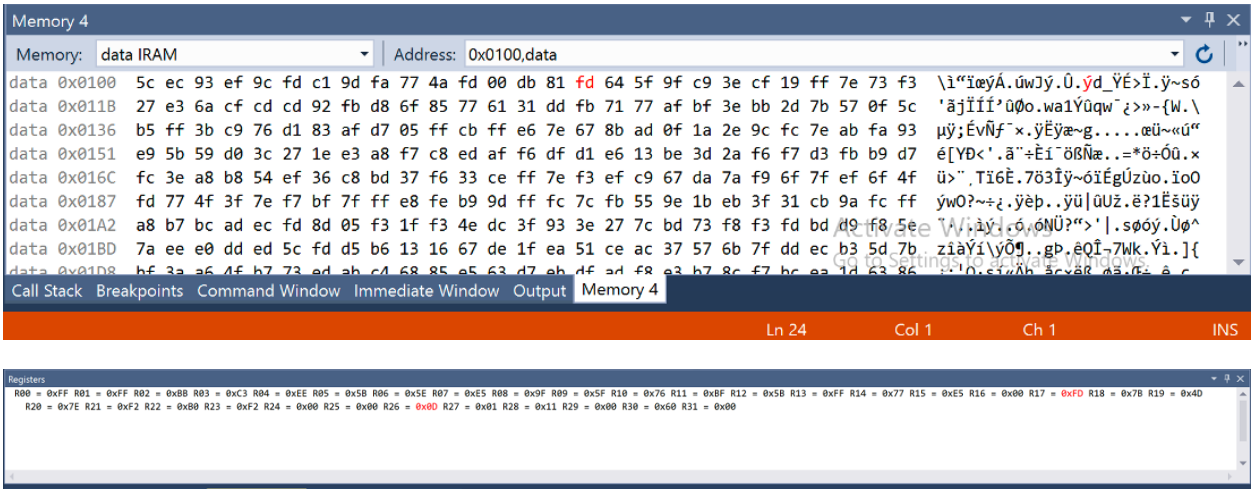


Figure 3. Output data of program 2, test 1. Max value is 0xFD.

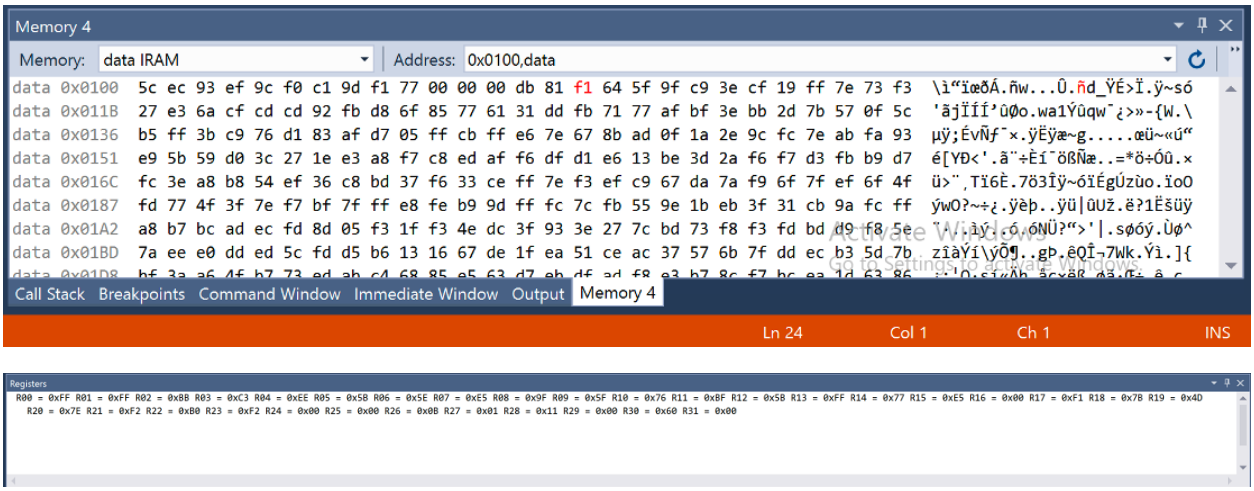


Figure 4. Output data of program 2, test 2. Max value is 0xF1.

Program 3 (is max value greater than twice the minimum)

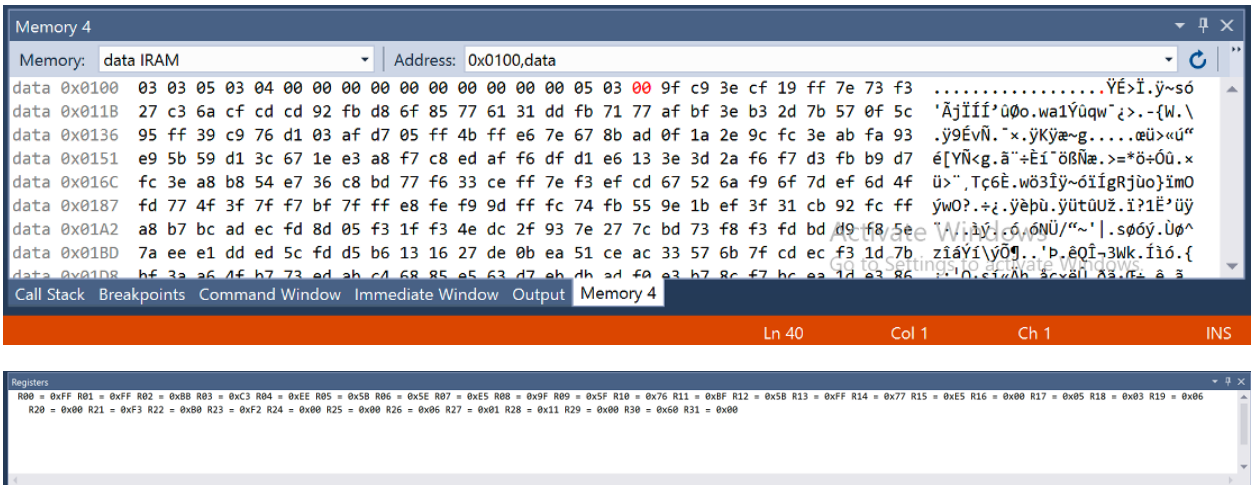
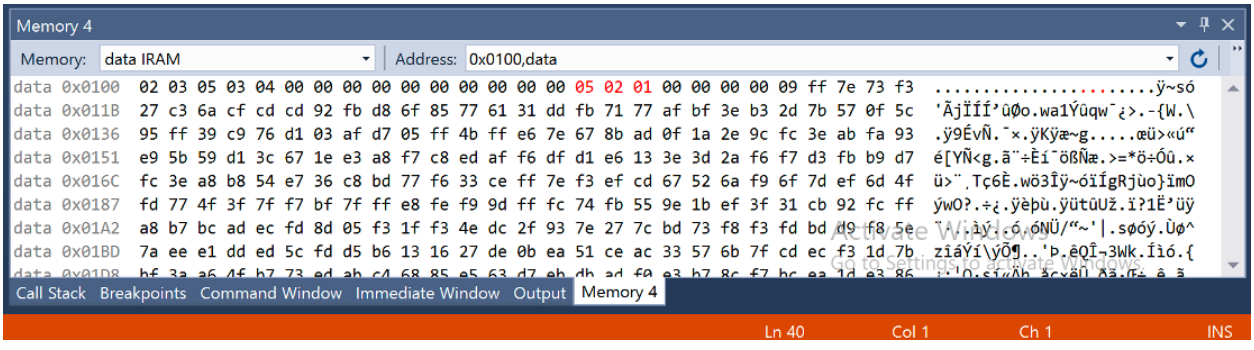


Figure 5. Output data of program 3, test 1. Output is at 0x112. Minimum value is 3, maximum value is 5, result is 0 (false).



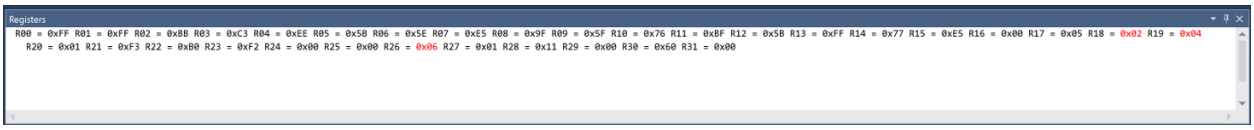


Figure 6. Output data of program 3, test 2. Output is at 0x112. Minimum value is 2, maximum value is 5, result is 1 (true).

Post-Lab

Program 1

Instruction	Clocks	Ran multiple times? (X = number of items in list)
jmp	2	
clr	1	
ldi	1	2
ld	2	X+1
cpi	1	X+1
breq (branch not taken)	1	X
breq (branch taken)	2	
ldi	1	X
add	1	X
rjmp	2	X
sts	2	

$$clocks = 5 + 3(x + 1) + 5x + 4$$

$$clocks = 9 + 3x + 3 + 5x$$

$$clocks = 12 + 8x$$

Given 15 items in list:

$$clocks = 12 + 8 * 15 = 132$$

Program 2

Instruction	Clocks	Ran multiple times? (X = number of items in list, Y = number of times max updated)
jmp	2	
clr	1	
ldi	1	2
ld	2	X+1
cpi	1	X+1

breq (branch not taken)	1	X
breq (branch taken)	2	
cp	1	X
brlo (branch not taken)	1	Y
brlo (branch taken)	2	X-Y
mov	1	Y
rjmp	2	X
sts	2	

$$clocks = 9 + 3(x + 1) + 4x + 2(x - y) + 2y$$

$$clocks = 9 + 3x + 3 + 4x + 2x - 2y + 2y$$

$$clocks = 12 + 9x$$

Given 15 items in list:

$$clocks = 12 + 9 * 15 = 147$$

Program 3

Instruction	Clocks	Ran multiple times? (X = number of items in list, Y = number of times max updated, Z = number of times min updated)
jmp	2	
clr	1	
ldi	1	3
ld	2	X+1
cpi	1	X+1
breq (branch not taken)	1	X
breq (branch taken)	2	
cp	1	X
brlo (branch not taken)	1	Y
brlo (branch taken)	2	X-Y
mov	1	Y
cp	1	X
brsh (branch not taken)	1	Z
brsh (branch taken)	2	X-Z

mov	1	Z
rjmp	2	X
sts	2	2
mov	1	
lsl	1	
cp	1	
ending	5 if max < 2*min 6 if max >= 2*min	

$$clocks = 20 + 1(if\ max >= 2 * min) + 3(x + 1) + 5x + 2y + 2(x - y) + 2z + 2(x - z)$$

$$clocks = 20 + 1(if\ max >= 2 * min) + 3x + 3 + 5x + 2y + 2x - 2y + 2z + 2x - 2z$$

$$clocks = 23 + 1(if\ max >= 2 * min) + 12x$$

Given 15 items in list & max >= 2*min

$$clocks = 23 + 1 + 12 * 15 = 204$$

So the worst case scenario for running all these programs once, given a clock rate of 16.00MHz and $0.000001/hour of processor time:

$$\frac{\$1 * 10^{-6}}{1\ hour} * \frac{1\ second}{16 * 10^{6}\ clocks} * (204 + 147 + 132\ clocks) * \frac{1\ hour}{3600\ seconds} = \$8.4 * 10^{-15}$$

By running this in Microchip Studio, we can also get a count of the clocks.

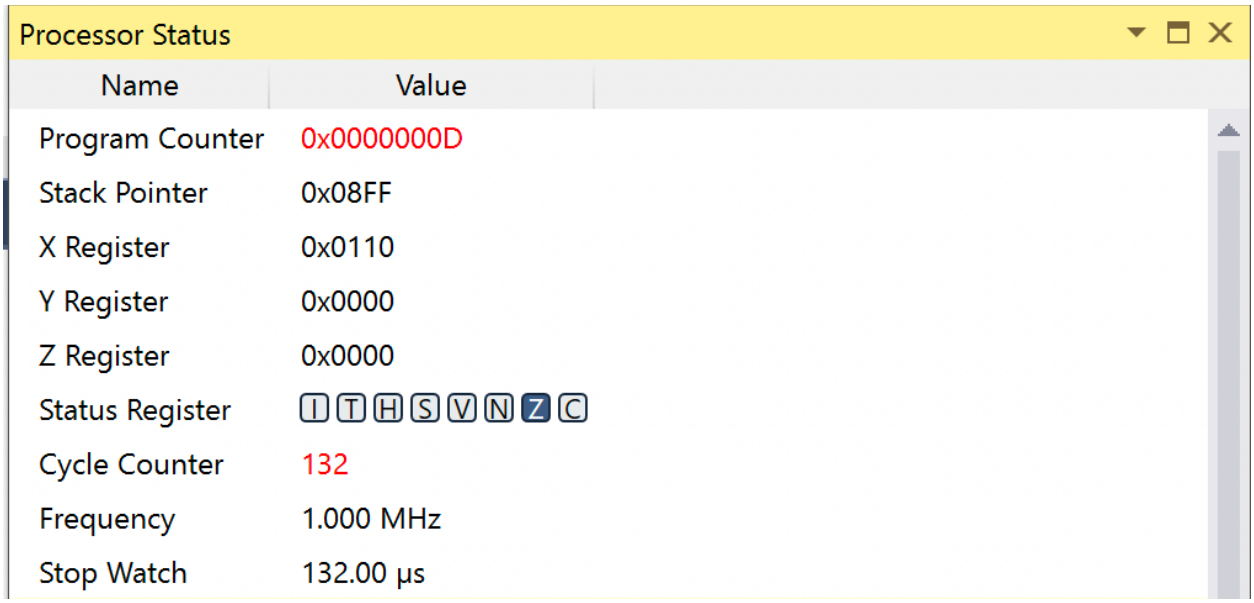


Figure 7. Clock count of program 1 with a 15-item list.



Figure 8. Clock count of program 2 with a 15-item list.



Figure 9. Clock count of program 3 with a 15-item list.

The calculated clock counts agree completely with the tested clock counts. Note that the number of times that max or min were updated do not contribute any difference to the total clock counts.

Discussion/Conclusion

This lab was successful in guiding the usage of Microchip Studio and the Atmel-ICE debugger. I was able to complete the in-lab procedure smoothly and understand the basic workings of a microcontroller.

Appendix A: Code

Program 1:

```
    .cseg
    jmp main                ; Jump to first real instruction

main:
    clr r17                 ; r17 = item counter, start at 0
    ldi XH, high(data)      ; Make X point to first byte of data
    ldi XL, low(data)

loop:
    ld r16, X+              ; Fetch next byte, bump index
    cpi r16, 0              ; Compare r16 to zero
    breq done               ; If r16 was zero then stop
    ldi r18, 1              ; Load the constant 1 into r18
    add r17, r18            ; Add 1 to our running count
    rjmp loop               ; Go check the next byte

done:
    sts count, r17          ; Store result

fin:
    rjmp fin

; Data starts here (On this processor this is always 0x100)
    .dseg
data:  .byte 15             ; 15 item list
count: .byte 1              ; Space for result (total number of items)
```

Program 2:

```
    .cseg
    jmp main            ; Jump to first real instruction

main:
    clr r17             ; running max
    ldi XH, high(data)  ; Make X point to first byte of data
    ldi XL, low(data)

loop:
    ld r16, X+           ; Fetch next byte, bump index
    cpi r16, 0           ; Compare r16 to zero
    breq done            ; If r16 was zero -> stop
    cp r16, r17          ; Compare current byte to running max
    brlo skip            ; If smaller, skip updating
    mov r17, r16         ; Otherwise, this is the new max
skip:
    rjmp loop            ; Go check the next byte


done:
    sts max, r17         ; Store result

fin:
    rjmp fin

; Data starts here (On this processor this is always 0x100)
    .dseg
data:  .byte 15         ; 15 item list
max: .byte 1           ; Space for result (total number of items)
```

Program 3:

```
    .cseg
    jmp main            ; Jump to first real instruction

main:
    clr r17             ; running max
    ldi r18, 0xFF       ; running min, start at highest possible byte
value
    ldi XH, high(data)  ; Make X point to first byte of data
    ldi XL, low(data)

loop:
    ld r16, X+          ; Fetch next byte, bump index
    cpi r16, 0           ; Compare r16 to zero
    breq done            ; If r16 was zero -> stop
    cp r16, r17          ; Compare current byte to running max
    brlo skipmax         ; If smaller, skip updating
    mov r17, r16         ; Update new max
skipmax:
    cp r16, r18          ; Compare current byte to running min
    brsh skipmin         ; If bigger or equal, skip updating
    mov r18, r16         ; Update new min
skipmin:
    rjmp loop            ; Go check the next byte


done:
    sts max, r17         ; Store max
    sts min, r18         ; Store min
    mov r19, r18         ; Copy min into r19
    lsl r19              ; r19 = min * 2 (shift left = multiply by 2)
    cp r17, r19          ; Compare max to (2 * min)
    brlo notgreater      ; If max < 2*min, skip
    ldi r20, 1           ; If max > 2*min, store 1
    rjmp storeresult
notgreater:
    ldi r20, 0           ; max > 2*min is false, store 0
storeresult:
    sts result, r20      ; Store the true/false result
```

```
fin:
    rjmp fin

; Data starts here (On this processor this is always 0x100)
    .dseg
data:   .byte 15        ; 15 item list
max:    .byte 1         ; Largest value found
min:    .byte 1         ; Smallest value found
result: .byte 1         ; 1 if max > 2*min, else 0
```